

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Криптография и сетевая безопасность»

Тема: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнила:

Студентка 4 курса

Группы АС-66

Прокурат В.Д.

Проверила:

Хацкевич А.С.

Цель работы: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Вариант 9

Общая схема выполнения (Индивидуальный вариант): задание выполняется строго в соответствии со своим вариантом. Все задачи объединены в единый программный комплекс (консольное приложение).

Этап 1. Генерация и тестирование псевдослучайных последовательностей. В рамках данного этапа необходимо разработать программный модуль генерации последовательностей и провести их первичный анализ. Требования:

- реализовать алгоритм генерации по варианту в виде программной функции/класса, способной выдавать последовательность произвольной длины;
- подобрать параметры генератора (a, c, N, x_0 и др.) так, чтобы добиться максимального периода последовательности $T_{\max}$ (согласно теоретическим условиям из методички);
- провести базовое статистическое тестирование полученной последовательности (например, оценить частотность появления элементов, равномерность распределения).

Генераторы Фибоначчи, размер алфавита $N=2$.

Этап 2. Тестирование на простоту и генерация больших простых чисел. На втором этапе сгенерированные на первом этапе псевдослучайные числа (или базовые механизмы генерации) интегрируются в модуль поиска и проверки чисел на простоту. Требования:

- реализовать стандартную схему генерации кандидата в простые числа:
 - сгенерировать случайное p -битовое число n (используя генератор из Этапа 1);
 - установить старший и младший биты в 1 (для гарантии разрядности и нечетности);
 - выполнить предварительную фильтрацию (просеивание) делением на малые простые числа (2, 3, 5, 7, ..., вплоть до 256 или 2000), что отсекает до 80% составных чисел без тяжелых тестов;
- интегрировать целевой тест из таблицы вариантов (вместо стандартного Рабина-Миллера или в дополнение к нему) для финальной проверки кандидата. (Примечание: для метода Михалеску используется собственная специфическая схема генерации доказуемо простых чисел).

Тест Рабина-Миллера.

main.py:

```
from __future__ import annotations

import subprocess
import sys

from experiments import run_all_experiments
from fibonacci import FibonacciGenerator, find_period, search_periods,
theoretical_max_period
from primes import generate_prime, miller_rabin
from stats import frequencies, run_all

def ask(prompt: str, default, cast=int):
    raw = input(f"{prompt} [{default}]: ").strip()
    return default if raw == "" else cast(raw)

def menu_sequence() -> None:
    r = ask("r", 10)
    s = ask("s (s < r)", 3)
    k = ask("k (размер алфавита N = 2^k, для варианта k=1)", 1)
    length = ask("длина последовательности", 64)
    gen = FibonacciGenerator(r, s, k)
    seq = gen.sequence(length)
    shown = "".join(map(str, seq)) if k == 1 else " ".join(map(str, seq))
    print(f"{gen}\nПоследовательность ({min(length, 200)} из {length} элементов:")
    print(shown[:400])
    print("Частоты символов:", frequencies(seq, gen.N))

def menu_period() -> None:
    r = ask("r", 10)
    res = search_periods(r)
    print(f"Теоретический максимум: {theoretical_max_period(r)}")
    for s, t in res:
        print(f" s={s:>2}: T={t}{ ' <- максимум' if t == 2 ** r - 1 else ''}")

def menu_stats() -> None:
    r = ask("r", 17)
    s = ask("s", 3)
    length = ask("длина последовательности", 20000)
    gen = FibonacciGenerator(r, s)
    seq = gen.sequence(length)
    print(f"{gen}, длина {length}, частоты {frequencies(seq, gen.N)}")
    for res in run_all(seq, gen.N):
        print(" ", res)

def menu_prime() -> None:
    bits = ask("разрядность p (бит, >= 16)", 256)
    limit = ask("просеивать малыми простыми < (0 - не просеивать, 256 или 2000)", 2000)
    seq_mode = ask("последовательный перебор вместо случайных кандидатов? (0/1)", 0) == 1
    gen = FibonacciGenerator(127, 1, 1, seed=int.from_bytes(__import__("os").urandom(16),
"big") | 1)
    gen.skip(500)
```

```

n, log = generate_prime(bits, gen, small_limit=limit, sequential=seq_mode)
print("\nПростое число:\n", n)
print("\nЛог:\n" + log.summary())

def menu_check() -> None:
    n = ask("число n", 561)
    rounds = ask("число раундов", 5)
    print("вероятно простое" if miller_rabin(n, rounds) else "составное")

def menu_tests() -> None:
    subprocess.run([sys.executable, "-m", "unittest", "-v", "test_lab"])

ITEMS = [
    ("Часть 1: сгенерировать последовательность", menu_sequence),
    ("Часть 1: найти периоды / подобрать параметры", menu_period),
    ("Часть 1: статистические тесты равномерности", menu_stats),
    ("Часть 2: сгенерировать простое число (с логом)", menu_prime),
    ("Часть 2: проверить число тестом Рабина-Миллера", menu_check),
    ("Отчёт: запустить все эксперименты (графики и таблицы)", run_all_experiments),
    ("Отчёт: запустить автотесты корректности", menu_tests),
]

def main() -> None:
    while True:
        print("\n=== Лабораторная работа №1 ===")
        for i, (title, _) in enumerate(ITEMS, 1):
            print(f"{i}. {title}")
        print("0. Выход")
        choice = input("Выбор: ").strip()
        if choice == "0":
            break
        if choice.isdigit() and 1 <= int(choice) <= len(ITEMS):
            try:
                ITEMS[int(choice) - 1][1]()
            except (ValueError, KeyboardInterrupt) as exc:
                print("Ошибка ввода:", exc)
        else:
            print("Неверный пункт меню")

if __name__ == "__main__":
    main()

```

```

Выбор: 1
r [10]:
s (s < r) [3]:
k (размер алфавита N = 2^k, для варианта k=1) [1]:
длина последовательности [64]:
FibonacciGenerator(r=10, s=3, k=1, N=2)
Последовательность (64 из 64 элементов):
00011100010011101100101011101110101000111101001001000001011111
Частоты символов: [30, 34]

```

Выбор: 2
 r [10]:
 Теоретический максимум: 1023
 s= 1: T=889
 s= 2: T=42
 s= 3: T=1023 <- максимум
 s= 4: T=62
 s= 5: T=15
 s= 6: T=62
 s= 7: T=1023 <- максимум
 s= 8: T=42
 s= 9: T=889

Выбор: 3
 r [17]:
 s [3]:
 длина последовательности [20000]:
 FibonacciGenerator(r=17, s=3, k=1, N=2), длина 20000, частоты [9940, 10060]
 Частотный (хи-квадрат): статистика=0.7200, df=1, p-value=0.3961 -> ПРОЙДЕН (alpha=0.05)
 Серийный (пары, хи-квадрат): статистика=3.1872, df=3, p-value=0.3637 -> ПРОЙДЕН (alpha=0.05)
 Серий (Вальд-Вольфовиц): статистика=2.2538, p-value=0.0242 -> НЕ ПРОЙДЕН (alpha=0.05)
 Автокорреляция (lag=1): статистика=-0.0160, p-value=0.0238 -> НЕ ПРОЙДЕН (alpha=0.05)

Выбор: 4
 разрядность p (бит, >= 16) [256]:
 просеивать малыми простыми < (0 - не просеивать, 256 или 2000) [2000]:
 последовательный перебор вместо случайных кандидатов? (0/1) [0]:
 Простое число:
 80534618929577592055653179721236902827839776181795068556482052523230728313357

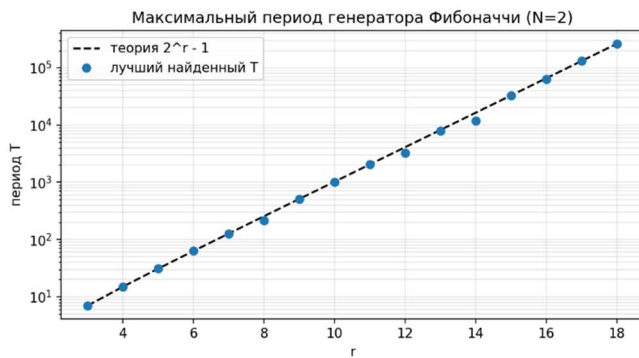
Лог:
 бит: 256, просеивание простыми < 2000, режим: новый случайный кандидат
 кандидатов проверено: 24
 отсеяно делением на малые простые: 23 (по первым простым -> 3: 6, 5: 6, 7: 2, 11: 2, 13: 1, 17: 1, 19: 1, 23: 1)
 отсеяно тестом Рабина-Миллера: 0
 время: 0.0032 c

Выбор: 5
 число n [561]:
 число раундов [5]:
 составное

Выбор: 6
 == Эксперимент 1.1. Подбор параметров (r, s) для N = 2 ==

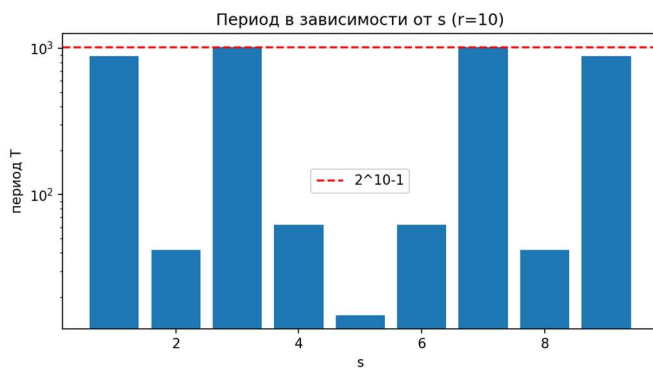
r	2^{r-1}	найденный max T	s с макс. периодом
3	7	7	[1, 2]
4	15	15	[1, 3]
5	31	31	[2, 3]
6	63	63	[1, 5]
7	127	127	[1, 3, 4, 6]
8	255	217	нет
9	511	511	[4, 5]
10	1023	1023	[3, 7]
11	2047	2047	[2, 9]
12	4095	3255	нет
13	8191	8001	нет
14	16383	11811	нет
15	32767	32767	[1, 4, 7, 8, 11, 14]
16	65535	63457	нет
17	131071	131071	[3, 5, 6, 11, 12, 14]
18	262143	262143	[7, 11]

[csv] results\exp1_best_params.csv
 [png] results\exp1_best_params.png



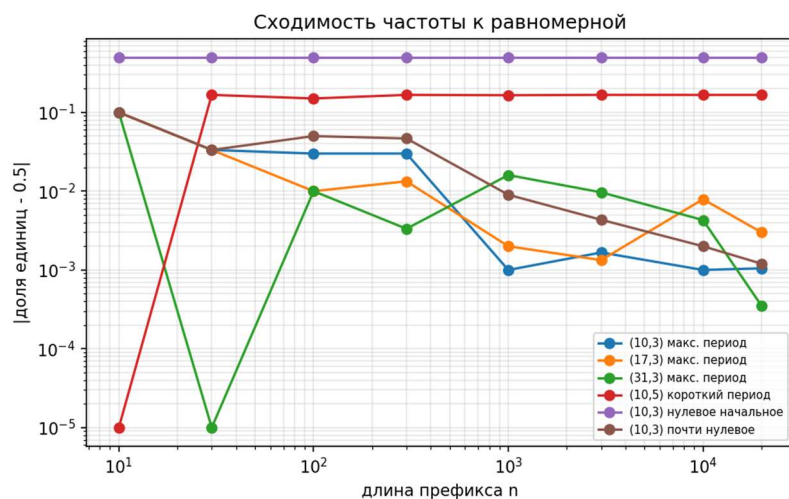
== Эксперимент 1.2. Период при $r = 10$ в зависимости от s ==

```
s= 1: T=889
s= 2: T=42
s= 3: T=1023 <- максимум
s= 4: T=62
s= 5: T=15
s= 6: T=62
s= 7: T=1023 <- максимум
s= 8: T=42
s= 9: T=889
[csv] results\exp2_period_vs_s_r10.csv
[png] results\exp2_period_vs_s_r10.png
```



== Эксперимент 1.3. Статистика vs параметры (длина 20000) ==

конфигурация | период T | доля 1 | ρ частотный | ρ серийный | ρ серий | ρ автокорр.
 (10,3) макс. период | 1023 | 0.501 | 0.7665 | 0.9694 | 0.7445 | 0.7392
 (17,3) макс. период | 131071 | 0.503 | 0.3961 | 0.3637 | 0.0242 | 0.0238
 (31,3) макс. период | 2147483647 | 0.4996 | 0.9211 | 0.6144 | 0.5338 | 0.5385
 (10,5) короткий период | 15 | 0.6665 | 0.0 | 0.0 | 0.0 | 0.0
 (10,3) нулевое начальное | 1 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0
 (10,3) почти нулевое | 1023 | 0.5012 | 0.7343 | 0.9825 | 0.8314 | 0.8259
 (p-value $\leq 0.05 \rightarrow$ гипотеза о равномерности/случайности отвергается)
 [csv] results\exp3_stat_vs_params.csv
 [png] results\exp3_frequency_convergence.png



== Эксперимент 1.4. Длина выборки относительно периода ==

```
(r,s) | T | длина L | p частотный | p серийный | p серий
(10,3) | 1023 | 100 | 0.5485 | 0.8685 | 0.399
(10,3) | 1023 | 1000 | 0.9496 | 0.1935 | 0.6126
(10,3) | 1023 | 10000 | 0.8415 | 0.9629 | 0.7036
(10,3) | 1023 | 100000 | 0.7138 | 0.9698 | 0.6993
(17,3) | 131071 | 100 | 0.8415 | 0.6414 | 0.3127
(17,3) | 131071 | 1000 | 0.8993 | 0.7453 | 0.0428
(17,3) | 131071 | 10000 | 0.1141 | 0.3007 | 0.0919
(17,3) | 131071 | 100000 | 0.8694 | 0.4945 | 0.5106
[csv] results\exp4_length_vs_period.csv
```

```
(r,s);T;длина L;p частотный;p серийный;p серий
(10,3);1023;100;0.5485;0.8685;0.399
(10,3);1023;1000;0.9496;0.1935;0.6126
(10,3);1023;10000;0.8415;0.9629;0.7036
(10,3);1023;100000;0.7138;0.9698;0.6993
(17,3);131071;100;0.8415;0.6414;0.3127
(17,3);131071;1000;0.8993;0.7453;0.0428
(17,3);131071;10000;0.1141;0.3007;0.0919
(17,3);131071;100000;0.8694;0.4945;0.5106
```

== Эксперимент 2.1. Генерация простых чисел (среднее по запускам) ==

```
бит | просеивание < | кандидатов | отсеяно делением | отсеяно P-M | доля пережив. просеивание | время, с
64 | нет | 16 | 0 | 15 | 1.0 | 0.0012
64 | 256 | 30.4 | 25 | 4.4 | 0.178 | 0.0011
64 | 2000 | 56.4 | 50.8 | 4.6 | 0.099 | 0.0017
128 | нет | 24.4 | 0 | 23.4 | 1.0 | 0.0026
128 | 256 | 45 | 34.2 | 9.8 | 0.24 | 0.0029
128 | 2000 | 89 | 76 | 12 | 0.146 | 0.0052
256 | нет | 76.8 | 0 | 75.8 | 1.0 | 0.0208
256 | 256 | 161.6 | 127.6 | 33 | 0.21 | 0.0221
256 | 2000 | 201.2 | 174 | 26.2 | 0.135 | 0.0249
512 | нет | 180.6 | 0 | 179.6 | 1.0 | 0.1767
512 | 256 | 175.4 | 137.8 | 36.6 | 0.214 | 0.0783
512 | 2000 | 162 | 139.6 | 21.4 | 0.138 | 0.0594
1024 | нет | 220.2 | 0 | 219.2 | 1.0 | 1.0535
1024 | 256 | 512.6 | 408.4 | 103.2 | 0.203 | 0.7586
1024 | 2000 | 433.8 | 375.2 | 57.6 | 0.135 | 0.5069
```

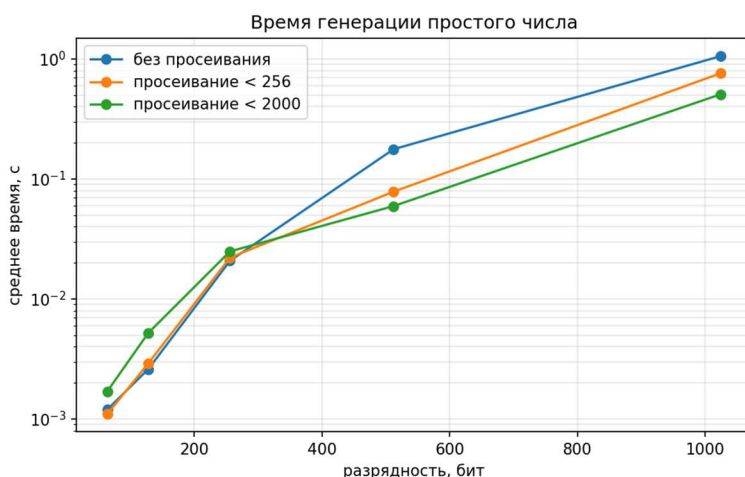
Теория: доля нечётных, не делящихся на простые $< z$, $\sim 1.12/\ln z$:

$z=256$: 0.202

$z=2000$: 0.147

[csv] results\exp5_prime_generation.csv

[png] results\exp5_prime_time.png



== Эксперимент 2.2. Случайный vs последовательный перебор (512 бит) ==

случайный: кандидатов=378.2, время=0.1382 с

последовательный: кандидатов=83.4, время=0.0186 с

[csv] results\exp6_sequential_vs_random.csv

```
режим;кандидатов;время, с
случайный;378.2;0.1382
последовательный;83.4;0.0186
```

```
Выбор: 7
test_alphabet (test_lab.TestFibonacci.test_alphabet) ... ok
test_max_period (test_lab.TestFibonacci.test_max_period) ... ok
test_non_max_period (test_lab.TestFibonacci.test_non_max_period) ... ok
test_periodicity_and_balance (test_lab.TestFibonacci.test_periodicity_and_balance) ... ok
test_recurrence (test_lab.TestFibonacci.test_recurrence) ... ok
test_stat_good_vs_bad (test_lab.TestFibonacci.test_stat_good_vs_bad) ... ok
test_big_numbers (test_lab.TestPrimes.test_big_numbers) ... ok
test_carmichael (test_lab.TestPrimes.test_carmichael) ... ok
test_generate_prime (test_lab.TestPrimes.test_generate_prime) ... ok
test_generate_prime_sequential (test_lab.TestPrimes.test_generate_prime_sequential) ... ok
test_miller_rabin_vs_bruteforce (test_lab.TestPrimes.test_miller_rabin_vs_bruteforce) ... ok
test_mod_pow (test_lab.TestPrimes.test_mod_pow) ... ok
test_sieve (test_lab.TestPrimes.test_sieve) ... ok
test_chi2_sf (test_lab.TestStatsMath.test_chi2_sf) ... ok

-----
Ran 14 tests in 0.160s

OK
```

Вывод: освоили программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.